R3.07
SQL dans un langage de programmation

Chapitre 2

Transactions et concurrence d'accès



Selena Lamari et Chouki Tibermacine

BUT 2 Informatique — IUT de Vannes — Université Bretagne Sud

Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Exigences pour les rendus de TP

Qu’est-ce qu’une transaction ?

Définition

Une **transaction** est une suite d’opérations sur la base qui forme une **unité logique de travail** : soit tout est appliqué (`COMMIT`), soit rien (`ROLLBACK`).

Propriétés ACID

- **A**tomicité : tout ou rien
- **C**ohérence : les contraintes restent vraies
- **I**solation : les transactions concurrentes ne se gênent pas (à quel point ?)
- **D**urabilité : un `COMMIT` survit à une panne

Exemple : virement

1. débiter le compte A
2. créditer le compte B
3. journaliser l’opération

Une panne entre 1 et 2 ne doit pas faire « disparaître » d’argent.

Transactions en JDBC : le schéma à retenir

```
try (Connection c = dataSource.getConnection()) {
  c.setAutoCommit(false);              // par defaut : autocommit apres CHAQUE requete
  try {
    debiter(c, src, montant);
    crediter(c, dst, montant);
    c.commit();                        // tout est valide
  } catch (SQLException e) {
    c.rollback();                      // rien n'est applique
    throw e;
  }
}                                      // close() rend la connexion au pool
```

- Toutes les requêtes d'une transaction utilisent la même `Connection`
- Points de reprise partiels possibles : `Savepoint sp = c.setSavepoint(); ... c.rollback(sp);`

Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Exigences pour les rendus de TP

Pourquoi la concurrence est inévitable

- Une application web ou mobile sert **plusieurs utilisateurs en même temps**
- Chaque requête HTTP est traitée par un **thread** différent, avec sa propre connexion (pool)
- Plusieurs transactions lisent et écrivent donc les **mêmes lignes** au même moment

Question

Deux guichets débitent le même compte (1000 €) au même instant : -100 et -50. Quel est le solde final ?

Pourquoi la concurrence est inévitable

- Une application web ou mobile sert **plusieurs utilisateurs en même temps**
- Chaque requête HTTP est traitée par un **thread** différent, avec sa propre connexion (pool)
- Plusieurs transactions lisent et écrivent donc les **mêmes lignes** au même moment

Question

Deux guichets débitent le même compte (1000 €) au même instant : −100 et −50. Quel est le solde final ? Pas forcément 850 €…

L’anomalie la plus courante : la mise à jour perdue



- Chaque transaction calcule le nouveau solde **dans le code Java** à partir d’une valeur lue trop tôt
- C’est le niveau d’isolation **par défaut** de PostgreSQL (`READ COMMITTED`) : testé, résultat 900

Les anomalies de lecture (norme SQL)

Anomalie	Ce qui se passe
Lecture sale	T1 lit une valeur modifiée par T2 **non encore validée** ; T2 fait ensuite `ROLLBACK`
Lecture non répétable	T1 lit deux fois la même ligne et obtient deux valeurs différentes (T2 a validé entre-temps)
Lecture fantôme	T1 relance une requête et voit des **lignes en plus ou en moins** (insertion/suppression de T2)
Mise à jour perdue	T1 et T2 lisent puis écrivent la même ligne : l'écriture de l'une écrase celle de l'autre

Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Exigences pour les rendus de TP

Les quatre niveaux d'isolation

Niveau	Lect. sale	Non répétable	Fantôme	Màj perdue
`READ UNCOMMITTED`	possible*	possible	possible	possible
`READ COMMITTED` (défaut PG)	—	possible	possible	possible
`REPEATABLE READ`	—	—	possible*	détectée
`SERIALIZABLE`	—	—	—	détectée

* selon la norme. PostgreSQL est plus strict : `READ UNCOMMITTED` s'y comporte comme `READ COMMITTED` (jamais de lecture sale), et `REPEATABLE READ` y empêche aussi les fantômes.

- Plus le niveau est élevé, plus il y a de transactions **refusées** ou **bloquées** : on paie l'isolation en débit
- PostgreSQL utilise le **MVCC** : chaque transaction lit une « photo » (snapshot) de la base ; les lecteurs ne bloquent jamais les écrivains

Choisir le niveau d'isolation

En SQL

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT solde FROM Compte
 WHERE numCompte = 1;
UPDATE Compte SET solde = 900
 WHERE numCompte = 1;
COMMIT;
```

En JDBC

```
c.setAutoCommit(false);
c.setTransactionIsolation(
  Connection.TRANSACTION_REPEATABLE_READ);
// ... requetes ...
c.commit();
```

Ce qu'on a observé sur PostgreSQL (scénario du guichet)

- `READ COMMITTED` : solde final 900 — mise à jour perdue
- `REPEATABLE READ` : T1 reçoit `ERROR: could not serialize access due to concurrent update` — SQLState 40001

⇒ à un niveau élevé, l'application doit **rejouer** la transaction refusée

Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Exigences pour les rendus de TP

Solution 1 : laisser la base faire le calcul

```java
// NON : lecture, calcul en Java, puis ecriture (2 requetes, fenetre de concurrence)
double solde = lireSolde(c, num);
majSolde(c, num, solde - montant);

// OUI : une seule requete, atomique pour la ligne concernee
PreparedStatement ps = c.prepareStatement(
  "UPDATE Compte SET solde = solde - ? WHERE numCompte = ? AND solde >= ?");
ps.setBigDecimal(1, montant); ps.setInt(2, num); ps.setBigDecimal(3, montant);
if (ps.executeUpdate() == 0) throw new SoldeInsuffisantException(num);
```

- La 2e transaction attend le verrou de ligne, puis relit la valeur à jour : résultat 850
- La condition `solde >= ?` vérifie la règle métier **dans la même requête**
- À privilégier dès que le calcul s'exprime en SQL

Solution 2 : verrou pessimiste (SELECT ... FOR UPDATE)

```
c.setAutoCommit(false);
PreparedStatement sel = c.prepareStatement(
  "SELECT solde FROM Compte WHERE numCompte = ? FOR UPDATE");  // pose un verrou
sel.setInt(1, num);
ResultSet rs = sel.executeQuery(); rs.next();
BigDecimal solde = rs.getBigDecimal("solde");
// ... regles metier complexes en Java ...
majSolde(c, num, solde.subtract(montant));
c.commit();                                                   // libere le verrou
```

- Les autres transactions qui veulent la même ligne **attendent** le COMMIT
- Utile quand la logique est trop complexe pour une seule requête
- Transaction **courte** obligatoire : jamais d'attente utilisateur pendant qu'on tient un verrou

Solution 3 : verrou optimiste (colonne de version)

```sql
ALTER TABLE Compte ADD COLUMN version INT NOT NULL DEFAULT 0;
-- lecture : on memorise la version lue (ex. 7)
SELECT solde, version FROM Compte WHERE numCompte = 1;
-- ecriture : seulement si personne n'a modifie la ligne entre-temps
UPDATE Compte SET solde = 900, version = version + 1
 WHERE numCompte = 1 AND version = 7;
```

- 0 ligne modifiée $\Rightarrow$ conflit : on prévient l'utilisateur ou on rejoue
- Aucun verrou tenu entre la lecture et l'écriture : adapté aux **longues interactions** (formulaire web)
- JPA l'automatise avec `@Version` (chapitre 6)

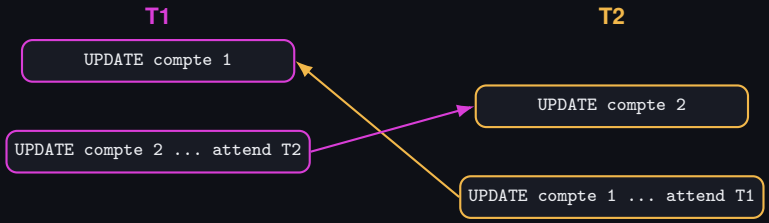
Quelle solution choisir ?

Approche	Quand l'utiliser	Prix à payer
Calcul en SQL	mise à jour simple (compteur, solde)	aucun
`FOR UPDATE`	logique complexe, conflits fréquents	attente, risque d'interblocage
Version (optimiste)	conflits rares, interaction longue	gérer l'échec (message, rejeu)
`REPEATABLE READ`/ `SERIALIZABLE`	règles portant sur plusieurs lignes	rejouer sur SQLState `40001`

Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Exigences pour les rendus de TP

Interblocage (*deadlock*)



- PostgreSQL détecte le cycle (au bout d'une seconde) et **annule l'une des deux** : `ERROR: deadlock detected` SQLState `40P01`
- Prévention : toujours verrouiller dans le **même ordre** (ex. compte de plus petit numéro d'abord), transactions courtes
- L'application traite `40P01` comme `40001` : on **rejoue** la transaction

Rejouer une transaction refusée

```
for (int essai = 1; ; essai++) {
  try (Connection c = dataSource.getConnection()) {
    c.setAutoCommit(false);
    c.setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ);
    try {
      virer(c, src, dst, montant);
      c.commit();
      return;
    } catch (SQLException e) {
      c.rollback();
      boolean rejouable = "40001".equals(e.getSQLState()) || "40P01".equals(e.getSQLState()
     );
      if (!rejouable || essai == 3) throw e;
    }
  }
}
```

Bonnes pratiques

- Des transactions **courtes** : pas d'appel réseau, pas d'attente utilisateur à l'intérieur
- La frontière de transaction est dans la **couche service** (chapitre 5), pas dans chaque DAO
- Préférer les mises à jour **relatives** en SQL (`solde = solde - ?`)
- Faire vérifier les règles par la base : `CHECK (solde >= 0)`, `UNIQUE`, clés étrangères
- Montants : `NUMERIC` en SQL, `BigDecimal` en Java (jamais `REAL`/`double`)
- Toujours `rollback()` en cas d'exception, et `close()` pour rendre la connexion au pool

Plan du cours

1. Transactions : rappels
2. Concurrence d’accès
3. Niveaux d’isolation
4. Éviter les mises à jour perdues
5. Interblocages
6. Exigences pour les rendus de TP

Un rendu de qualité, c'est…

Dépôt Git

- Un dépôt par binôme, commits réguliers et messages explicites
- `.gitignore` : `target/`, `.idea/`, `.env`
- Aucun mot de passe dans le code ni dans l'historique
- Projet Maven standard (`src/main/java`, `src/test/java`)

README.md

- Comment lancer la base (`docker compose up -d`)
- Comment compiler et exécuter (`mvn ...`)
- Ce qui est fait, ce qui ne l'est pas
- Choix techniques et difficultés

Test d'acceptation

Un enseignant clone le dépôt sur une machine vierge et suit le README : **ça doit fonctionner**.

Questions ?



Chouki.Tibermacine@univ-ubs.fr